VELLORE INSTITUTE OF TECHNOLOGY (VIT)

Course: Software Configuration Management

Course Code: ISWE403L

PROJECT REPORT

AI-Powered Document Question-Answering System Using Retrieval-Augmented Generation (RAG) with Fossil SCM

Faculty: KRITHIKA L.B

Slot: E2 + TE2

Team Members

Reg. No.	Name	University e-mail
24MIS0248	HARISANKAR C MOOSS	harisankar.cmooss2024@vitstudent.ac.in
24MIS0249	ACHYUTH P N	achyuth.pn2024@vitstudent.ac.in
24MIS0450	NIRANJAN SANTHOSH	niranjan.santhosh2024@vitstudent.ac.in

Submission date: [to be added]

Abstract

Large document collections are difficult to search manually, and keyword search does not capture the meaning of a question. This project builds DocuMind, an AI-powered document question-answering system that uses Retrieval-Augmented Generation (RAG): uploaded documents are processed and embedded, the passages most relevant to a question are retrieved by similarity search, and a locally hosted language model generates an answer grounded in them. The application uses Java and Spring Boot, a React/TypeScript frontend built with Vite, PostgreSQL with pgvector, Ollama (llama3.1:8b for chat and nomic-embed-text for embeddings), Flyway migrations and Docker Compose.

Because the system is developed by a team across Windows and Linux, it is managed with Fossil SCM. The report explains Fossil's architecture and how it was used in this project: a central repository with role-based accounts, cross-platform collaboration, a branch and merge demonstration, tickets, tags, wiki and configuration management. Following faculty feedback, the team built a custom, partly dynamic Fossil dashboard and integrated Fossil email alerts with Python and Gmail SMTP for the administrator. The report distinguishes practices that were planned from those that were demonstrated, classifies each Fossil limitation as overcome, improved or remaining, compares Fossil fairly with Git and the Git-based platforms GitHub and GitLab, and concludes that Fossil was a suitable choice for a project of this size. No quantitative RAG performance results are claimed.

Keywords: Software Configuration Management, Fossil SCM, Retrieval-Augmented Generation, pgvector, version control, email notifications.

Table of Contents

1. Introduction....1
1.1 Background and Motivation....1
1.2 Problem Statement....1
1.3 Organisation of the Report....1
2. Objectives, Scope and Evidence Convention....3
2.1 Objectives....3
2.2 Scope....3
2.3 Evidence Convention....3
3. Software Configuration Management and Fossil SCM....5
3.1 What Software Configuration Management Means....5
3.2 Why SCM Is Necessary in This Project....5
3.3 Introduction to Fossil SCM....5
3.3.1 Core Concepts....6
3.3.2 Web Interface and Administration....6
3.4 Rationale for Selecting Fossil SCM....6
4. The DocuMind RAG Application....8
4.1 Functional Overview....8
4.2 Retrieval-Augmented Generation....8
4.3 Technology Stack....8
4.4 System Architecture....9
4.5 The RAG Pipeline....9
4.6 Frontend and Backend Roles....10
4.7 Application Screens....11
4.8 How SCM Supports the Development and Maintenance of DocuMind....12
5. Fossil Repository Setup and Configuration....13
5.1 Environments....13
5.2 Repository Topology....13
5.3 The Default Interface Before Customisation....13
5.4 User Accounts and Permissions....14
6. Version Control and Change Management....16
6.1 Distinguishing the Basic Operations....16
6.2 Inspecting State and Change....17
6.3 Commit Messages in Practice....17
6.4 Timeline and Change History....17
7. Branching, Merging and Rollback....18
7.1 Branches in Fossil....18
7.2 The feature-demo Branch and Merge....18
7.3 Rollback and Recovery....20
8. Tickets, Tags and Wiki....21
8.1 Ticket Management....21
8.2 Tags and Release Identification....21
8.3 Wiki and Project Documentation....22
9. Cross-Platform Team Collaboration....24
9.1 Objective....24
9.2 Analysis....24
9.3 Server Availability....25
10. Custom DocuMind Fossil Dashboard....27
10.1 Motivation....27
10.2 Design Approach....27
10.3 Implemented Sections....27
10.3.1 Dynamic Content....28
10.4 Evidence and Analysis....28
10.5 Limitations of the Dashboard....31

11. Fossil Email Notification Integration....33
11.1 Requirement....33
11.2 Fossil Alerts and the Chosen Architecture....33
11.3 Configuration....33
11.4 Testing and Verification....34
11.5 Automatic Background Processing....37
12. Configuration Management, Security and Backup....38
12.1 What Is Versioned and What Is Not....38
12.2 Access Control and Transport Security....38
12.3 Backup and Recovery....38
13. Review History and Faculty Feedback....40
13.1 Review 1 and Review 2....40
13.2 Plan Versus Evidence....40
13.3 Feedback and Response....40
14. Comparison Study....41
14.1 Comparing Different Categories Fairly....41
14.2 Characteristics Compared....41
14.3 Fit for the DocuMind Project....42
15. Fossil Advantages, Limitations and Practical Mitigation....43
15.1 Observed Advantages....43
15.2 Limitations and What Was Done About Them....43
15.3 Summary of Unresolved Items....44
16. Testing and Results....45
16.1 Verification of the SCM Implementation....45
16.2 Testing of the DocuMind Application....45
16.3 Discussion....46
17. Team Contributions....47
18. Future Enhancements....48
19. Conclusion....49
References....50
Appendix A: Selected Fossil Commands....51
Appendix B: Screenshot Evidence Index....52

List of Figures

Figure 1. Architecture of DocuMind showing the frontend, backend, PostgreSQL/pgvector and Ollama components. [Diagram to be supplied by the team.]....9
Figure 2. DocuMind application home page....11
Figure 3. Document upload and processing screen....11
Figure 4. A question answered by DocuMind from an uploaded document....12
Figure 5. Original Fossil web interface before customisation, showing the default project name and the built-in site-map menu (Fossil 2.28)....14
Figure 6. Fossil user list (Admin → Users) showing the capability categories and the accounts Niran, OMEN and achyuth....15
Figure 7. Fossil timeline showing the initial repository history....17
Figure 8. Creation of the feature-demo branch and its feature check-in....19
Figure 9. Merge of feature-demo into trunk, preserved in the repository history....19
Figure 10. Fossil ticket management used to record project issues....21
Figure 11. Tags used to identify releases and milestones....22
Figure 12. Wiki pages used for DocuMind project documentation....22
Figure 13. Windows side of the cross-platform collaboration....25
Figure 14. Linux side of the cross-platform collaboration....25
Figure 15. Customised DocuMind Fossil dashboard showing the navigation bar, System Status and Project Information sections....28
Figure 16. Quick Actions section linking to the timeline, files, branches, tickets and documentation. 29
Figure 17. DocuMind Services and RAG Workflow sections of the custom dashboard....29
Figure 18. Documentation section of the dashboard....30
Figure 19. Repository Activity navigation cards (timeline, files, branches, tickets, wiki)....30
Figure 20. Dynamic Repository Overview and Repository Status sections....31
Figure 21. Project Team, dynamic Recent Activity and Project Version sections....31
Figure 22. Fossil email settings used for the Python pipe integration (no secrets shown)....35
Figure 23. Test message sent through the Python script and Gmail SMTP and received in the inbox. 35
Figure 24. Fossil check-in notification received by the administrator....36
Figure 25. Fossil ticket-change notification received by the administrator....36
Figure 26. Administrator subscriber and topic configuration....37
Figure 27. Testing evidence for the DocuMind application....46

List of Tables

Table 1. Report organisation by part....1
Table 2. Categories of statements used in this report....3
Table 3. SCM activities and how they were realised in the DocuMind project....5
Table 4. Fossil characteristics and their technical basis....6
Table 5. Selection factors, project fit and accepted trade-offs....7
Table 6. Confirmed technology stack and the role of each component....8
Table 7. RAG pipeline stages....9
Table 8. Pipeline parameters to be completed by the team....10
Table 9. Fossil environment on the two platforms....13
Table 10. User accounts visible in the user list....14
Table 11. Fossil operations: what each changes and where....16
Table 12. The feature-demo demonstration....18
Table 13. Rollback and recovery techniques....20
Table 14. Verified cross-platform collaboration sequence....24
Table 15. Dashboard sections....27
Table 16. Notification chain....33
Table 17. Fossil email settings (non-secret values only)....33
Table 18. Notification types and their status....34
Table 19. Configuration items and handling policy....38
Table 20. Review 1 plan compared with the evidence in this report....40
Table 21. Faculty feedback and the team’s response....40
Table 22. Comparison of Fossil, Git, GitHub and GitLab....41
Table 23. Limitations, actions taken and classification....43
Table 24. SCM verification summary....45
Table 25. Measurements not yet collected....45
Table 26. Functional test cases for DocuMind (results to be recorded by the team)....46
Table 27. Principal documented contributions....47
Table 28. Commands used....51
Table 29. Figure register....52

1. Introduction

1.1 Background and Motivation

Organisations, educational institutions and research groups accumulate large volumes of reports, manuals, policies, textbooks and research papers. Locating a specific piece of information in such a collection is slow when it relies on manual reading. Traditional keyword search improves on this only partially: it matches the words in a query, not the intent or context behind them, so a user must often open several documents and read around the matches before finding a useful answer.

DocuMind is an AI-powered document question-answering system built to address this difficulty. Users upload documents and ask questions in natural language; the system processes the documents, retrieves the passages most relevant to a question, and uses a language model to generate an answer grounded in the uploaded material. The technique is known as Retrieval-Augmented Generation (RAG) [11].

DocuMind is developed by a team of three students, and it consists of several interacting components: a Spring Boot backend, a React/TypeScript frontend, a PostgreSQL database with the pgvector extension, and locally hosted language models served by Ollama. Several people change these components concurrently, on different operating systems, and the components must remain mutually consistent (for example, a change to the embedding model affects the database schema and the retrieval code). Version control, change tracking, branching and merging, issue management, documentation and configuration management are therefore essential to the project and are the subject of this report, which is written for the course Software Configuration Management (ISWE403L).

1.2 Problem Statement

Finding relevant information in large document collections by manual searching or keyword matching is time-consuming and unreliable, because keyword search does not capture the meaning of a question. A system is needed that lets users query their own documents in natural language and receive answers that are based on those documents. Building such a system as a team introduces a second problem: the source code, database migrations, configuration files and documentation of a multi-component AI application must be developed collaboratively without losing history, overwriting each other's work or exposing secrets. This project addresses both problems: it builds the DocuMind RAG application, and it manages that application with Fossil SCM, extending Fossil where its defaults did not meet the team's needs.

1.3 Organisation of the Report

The report is organised in three parts that reflect the intended emphasis of roughly 40% on Fossil SCM, 30% on the DocuMind application and 30% on improvements, evaluation and comparison. The percentages describe content emphasis, not exact page counts.

Table 1. Report organisation by part

Part	Emphasis	Chapters	Content
I	Fossil SCM (≈40%)	3, 5, 6, 7, 8, 9, 12	SCM concepts, Fossil architecture and selection, repository setup, permissions, version control, branching and merging, tickets/tags/wiki, cross-platform collaboration, configuration management and backup
II	DocuMind RAG application (≈30%)	4, 16.2	Problem context, technology stack, architecture, RAG pipeline, application testing, and how SCM supports its development

Part	Emphasis	Chapters	Content
III	Improvements, evaluation and comparison (≈30%)	10, 11, 13, 14, 15, 18	Faculty feedback, custom dashboard, email notifications, comparison with Git/GitHub/GitLab, limitations and mitigation, future work

Chapters 1, 2, 16, 17 and 19 are common to all parts. Chapter 2 also defines the evidence convention used throughout the report.

2. Objectives, Scope and Evidence Convention

2.1 Objectives

- Build a working RAG-based document question-answering application (DocuMind) using Java/Spring Boot, React/TypeScript, PostgreSQL with pgvector, and Ollama-hosted models.
- Use Fossil SCM as the single source-control and project-management system for the team, covering version history, branching and merging, tickets, tags, wiki documentation and configuration management.
- Demonstrate collaboration between Windows and Linux team members through a central Fossil repository with role-based user accounts.
- Respond to faculty feedback by making the Fossil interface more interactive (a custom dashboard) and by adding administrator notifications (Fossil email alerts delivered through Python and Gmail SMTP).
- Identify the practical limitations of Fossil in this project, implement mitigations where possible, and state honestly which limitations remain.
- Compare Fossil with Git and with the Git-based platforms GitHub and GitLab, and justify the selection of Fossil for this project.

2.2 Scope

The scope covers the use of Fossil across the DocuMind development lifecycle, the architecture and operation of the DocuMind application, the dashboard and notification customisations, and a documentation-based comparison with Git-based tooling. The following are outside the scope: production deployment of DocuMind, quantitative benchmarking of the RAG pipeline (no measurements were collected, and none are reported), and any empirical trial of Git, GitHub or GitLab on this project. The comparison in Chapter 14 is therefore based on official documentation and on the team's hands-on experience with Fossil only.

2.3 Evidence Convention

Because some SCM practices were planned in Review 1 while others were actually demonstrated, every claim in this report belongs to one of the categories in Table 2. Statements about planned practice are never presented as demonstrated results.

Table 2. Categories of statements used in this report

Category	Meaning	How it is supported
Demonstrated	Carried out by the team and observable in the repository, a screenshot or a test	Figure or check-in reference
Configured	Set up by the team but not individually tested or evidenced	Stated as configured; test status given
Planned	Intended in Review 1 but not verified as carried out	Labelled "planned"
Documented	A fact about Fossil or another tool taken from external documentation	Numbered reference [n]
Recommendation	Good practice proposed by the authors, not yet implemented	Labelled "recommended"

> *TEAM NOTE (delete before submission): Yellow-highlighted bracketed text such as this marks information the team must confirm or fill in. Search for "[" and "TEAM" before submission.*

3. Software Configuration Management and Fossil SCM

3.1 What Software Configuration Management Means

Software Configuration Management (SCM) is the discipline of identifying the items that make up a software system, controlling changes to them, recording and reporting their status, and verifying that the system corresponds to what was intended [20]. Its classical activities are configuration identification, change control, status accounting and audit. A version-control tool supports these activities but is only one part of SCM: tickets, release identification, documentation and the handling of configuration data are equally part of the discipline.

Table 3. SCM activities and how they were realised in the DocuMind project

SCM activity	Purpose	Realisation with Fossil in this project
Configuration identification	Name and version every item under control	Each check-in receives a unique hash identifier; tags and branch names identify milestones and lines of development
Change control	Make changes deliberately and traceably	Check-ins with messages, branches for isolated work, merge check-ins, tickets for requested changes
Status accounting	Report the state of the configuration	Timeline, repository statistics, and the custom dashboard’s overview and status sections
Audit and review	Verify what was changed, by whom and when	Check-in author and time in the timeline, diffs, and email alerts to the administrator
Release management	Identify stable versions	Tags (planned; evidence to be confirmed, see Section 8.2)
Configuration data handling	Keep build/runtime configuration consistent and secrets out of history	Configuration files and dependency lists versioned; secrets supplied through environment variables (Chapter 12)

3.2 Why SCM Is Necessary in This Project

DocuMind is not a single program. A change to the embedding model (nomic-embed-text) alters the dimension of the vectors stored in PostgreSQL, which requires a database migration (Flyway), which in turn affects the retrieval code in the backend and possibly the frontend. Three developers work on these layers concurrently, on at least two operating systems (Windows and Linux). Without SCM the team would face overwritten work, an inability to identify which version produced a given behaviour, no safe way to return to a working state after a failed experiment, and no record of why a change was made. Because the project also depends on credentials (for example, mail credentials for notifications), it needs a disciplined way to keep secrets out of the shared history.

3.3 Introduction to Fossil SCM

Fossil is an open-source distributed version-control system that combines source control with bug tracking, a wiki, a forum, chat and a built-in web interface in a single self-contained executable [1][2]. It was created by D. Richard Hipp, the author of SQLite, and is used to manage the SQLite project itself [1]. The characteristics identified in Review 1 are summarised, with the technical reasons behind them, in Table 4.

Table 4. Fossil characteristics and their technical basis

Characteristic	Technical basis	Relevance to DocuMind
Single-file repository	The repository is a SQLite database stored in one file [1]	The central repository is one file (RAG-Document-QA.fossil), easy to copy, move and back up
Single executable	One portable program provides the command line, the web server and the web UI [1]	Installed on Windows (fossil.exe) and Linux (/usr/bin/fossil) without a separate server stack
Distributed	Each clone holds the full history; changes move between repositories by synchronisation [2]	Teammates can commit locally even when the central server is unreachable
Integrated tools	Tickets, wiki, forum, chat, timeline and file browser are part of the same program [1]	No separate issue tracker or wiki hosting needed (the site map in the default UI lists them, Figure 5)
Content-addressed storage	Artifacts are identified by cryptographic hash; the check-in ID is a hash of its manifest [3]	The check-in IDs shown on the dashboard are 64-hex-character hashes
Cross-platform	Builds for Windows, Linux and macOS [1]	Windows–Linux collaboration was demonstrated (Chapter 9)

3.3.1 Core Concepts

A **repository** is the database holding all history. A **checkout** (working directory) is a set of ordinary files extracted from one version of the repository on which a developer works; in this project the Windows working directory is C:\RAG-Document-QA. A **check-in** (commit) records a new version of the tracked files together with the author, time and message. An **artifact** is any immutable object stored in the repository (file content, check-in manifest, wiki page, ticket change). A **branch** is a named line of development, and a **tag** is a label attached to a check-in or branch. **Synchronisation** exchanges artifacts between two repositories (push and pull). Using consistent terminology matters because Fossil separates the repository from the checkout more explicitly than many users expect, and confusing the two causes most beginner errors (Chapter 6).

3.3.2 Web Interface and Administration

Running `fossil ui` or `fossil server` starts an HTTP server that exposes the timeline, file browser, branches, tags, tickets, wiki, forum, chat, user administration, skins and settings [1]. This web interface is where the team administered user accounts (Figure 6) and where the custom dashboard in Chapter 10 is displayed. The same server also answers synchronisation requests from remote clones, so one process serves both the browser and the collaborators.

3.4 Rationale for Selecting Fossil SCM

Fossil was selected in Review 1 because the project spans several development stages (document processing, the RAG pipeline, the user interface and AI-model integration) that need version history, issue tracking and documentation, and because a team of three on a course project benefits from a tool that provides all of these without external accounts or hosting. Table 5 relates each selection factor to the project and states the trade-off accepted. A fuller comparison with Git, GitHub and GitLab follows in Chapter 14.

Table 5. Selection factors, project fit and accepted trade-offs

Factor	Why it mattered here	Trade-off accepted
Integrated tickets, wiki, timeline	Bugs such as PDF upload errors, chunking and retrieval quality needed tracking next to the code	Less polished UI and workflow than commercial platforms
Self-hosting with one binary and one file	No dependence on an external service for the course project; simple installation on both operating systems	Availability depends on the team's own host (Section 9.3)
Offline-capable distributed model	Members can commit without the server	A reachable server is still needed to push and pull
Course relevance	The course studies SCM tools; a less common tool with built-in project management supports a richer study	Smaller community and tool ecosystem than Git
Customisable skins and external scripting	Allowed the dashboard (Chapter 10) and the Gmail notification pipe (Chapter 11) without modifying Fossil's core	Customisation is the team's responsibility

4. The DocuMind RAG Application

4.1 Functional Overview

DocuMind lets a user upload documents, have them processed into a searchable form, and ask natural-language questions whose answers are generated from the uploaded content. The RAG application is fully working as confirmed by the team. The functional flow is: document upload, text extraction, chunking, embedding generation, storage of vectors, retrieval of the passages most similar to a question, and answer generation by a language model that receives those passages as context. The Fossil dashboard's RAG Workflow section presents the same flow to the team (Chapter 10).

4.2 Retrieval-Augmented Generation

A language model on its own answers from what it learned during training and cannot see a user's private documents; it may also produce fluent but unsupported statements. RAG combines a retrieval component, which finds relevant text in an external collection, with a generation component, which conditions its answer on the retrieved text [11]. Compared with keyword search, retrieval is performed on vector embeddings so that passages with similar meaning can be found even when the wording differs. Compared with retraining or fine-tuning a model on every new document, RAG needs only indexing, so new documents become usable immediately. Grounding the answer in retrieved passages is intended to make answers traceable to the source material, although it does not guarantee correctness; answer quality still depends on retrieval quality.

4.3 Technology Stack

Table 6. Confirmed technology stack and the role of each component

Layer	Technology	Role in DocuMind
Backend	Java, Spring Boot [14]	REST API, document processing orchestration, calls to the database and to Ollama
Build	Maven	Dependency management and build of the backend
Frontend	React with TypeScript, built with Vite [21]	User interface for upload and question answering
Relational database	PostgreSQL [13]	Stores application data, including document and chunk records
Vector storage	pgvector extension [12]	Stores embedding vectors and performs similarity search inside PostgreSQL
Language model runtime	Ollama [15]	Hosts the models locally so documents and questions stay on the team's machines
Chat model	llama3.1:8b	Generates the answer from the retrieved context
Embedding model	nomic-embed-text	Converts chunks and questions into vectors
Schema migrations	Flyway [16]	Versioned, repeatable database schema changes kept in the repository
Containerisation	Docker and Docker Compose [17]	Reproducible configuration of the supporting services
Source control	Fossil SCM 2.28 [1]	Version history, branches, tickets, wiki and notifications

A single database engine for both relational data and vectors is a deliberate design simplification: PostgreSQL with pgvector avoids operating a separate vector database, keeps document records and their embeddings in one transactional store, and lets Flyway version the vector schema together with the rest of the schema.

> *TEAM NOTE (delete before submission): The dashboard screenshot of the Services and RAG Workflow sections currently labels the vector store "Qdrant". The application uses PostgreSQL with pgvector. Correct the label in the skin and re-capture that figure so the report and the dashboard agree (see Figure in Section 10.4).*

4.4 System Architecture

The architecture has three tiers plus a model runtime. The React/TypeScript frontend, built with Vite, sends requests to the Spring Boot backend over HTTP. The backend persists documents and chunks in PostgreSQL, stores and queries embeddings through pgvector, and calls Ollama to generate embeddings (nomic-embed-text) and answers (llama3.1:8b). Flyway applies versioned migrations to the database at deployment, and Docker Compose configuration describes the supporting services. All components of the RAG pipeline run in the team's own environment; no document content is sent to an external AI service.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 1 — RAG system architecture diagram

Required image: A diagram of the DocuMind architecture: React/TypeScript frontend (Vite) → Spring Boot backend → PostgreSQL with pgvector, and backend ↔ Ollama (llama3.1:8b and nomic-embed-text), with Flyway and Docker Compose indicated.

Evidence from: Team (draw in a diagramming tool; this is a design diagram, not captured evidence)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 1. Architecture of DocuMind showing the frontend, backend, PostgreSQL/pgvector and Ollama components. [Diagram to be supplied by the team.]

4.5 The RAG Pipeline

The pipeline has an ingestion path, executed once per document, and a query path, executed for every question. Table 7 describes each stage and the technology involved. Stage parameters that the project details do not specify are left as marked placeholders rather than assumed.

Table 7. RAG pipeline stages

Path	Stage	Description	Technology
Ingestion	1. Upload	User submits a document (the dashboard lists PDF and other document files) through the frontend	React frontend → Spring Boot API

Path	Stage	Description	Technology
Ingestion	2. Text extraction	Readable text is extracted from the uploaded file	Backend
Ingestion	3. Chunking	Text is split into passages small enough to embed and retrieve precisely	Backend
Ingestion	4. Embedding	Each chunk is converted into a vector	Ollama, nomic-embed-text
Ingestion	5. Storage	Chunk text and vector are stored; the vector column is indexed for similarity search	PostgreSQL + pgvector, schema by Flyway
Query	6. Question embedding	The user's question is embedded with the same model	Ollama, nomic-embed-text
Query	7. Retrieval	The chunks whose vectors are nearest to the question vector are selected	pgvector similarity search
Query	8. Generation	The question and retrieved chunks are sent to the language model as a prompt	Ollama, llama3.1:8b
Query	9. Answer	The generated answer is returned to the frontend	Spring Boot → React

Table 8. Pipeline parameters to be completed by the team

Parameter	Value
Chunk size and overlap	[TEAM TO FILL from source code/configuration]
Number of chunks retrieved (top-k)	[TEAM TO FILL]
Distance metric used by pgvector (L2, inner product or cosine)	[TEAM TO FILL]
Embedding dimension in the Flyway migration	[TEAM TO FILL; confirm against the nomic-embed-text model [15]]
Model client (e.g. Ollama REST API called directly, or a Spring library)	[TEAM TO FILL]

4.6 Frontend and Backend Roles

The frontend is responsible only for presentation and interaction: choosing files to upload, showing processing status, accepting a question and displaying the answer. The backend owns all logic that must be consistent and secure: text extraction, chunking, calls to Ollama, persistence, and the retrieval query. Keeping this boundary clean has an SCM benefit as well, since frontend and backend changes can be developed and reviewed independently and conflict less often in the repository.

4.7 Application Screens

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 2 — DocuMind application home page

Required image: Screenshot of the DocuMind web application's home/landing screen.

Evidence from: DocuMind application

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 2. DocuMind application home page.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 3 — Document upload and processing

Required image: Screenshot showing a document being uploaded and processed (or listed as processed).

Evidence from: DocuMind application

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 3. Document upload and processing screen.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 4 — Question-answering interaction

Required image: Screenshot of a question typed by the user and the generated answer.

Evidence from: DocuMind application

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 4. A question answered by DocuMind from an uploaded document.

4.8 How SCM Supports the Development and Maintenance of DocuMind

- **Coordinated change across layers.** Backend, frontend, migrations and Docker configuration live in one repository, so one check-in can record a change that spans layers and the timeline shows them together.
- **Versioned schema.** Flyway migration files are source files; their history in Fossil documents how the vector schema evolved and allows a schema to be matched to the code version that needs it.
- **Reproducibility.** Build files (Maven), frontend dependency lists and Docker Compose configuration under version control allow another member, on Windows or Linux, to reproduce the environment from a checkout.
- **Safe experimentation.** Model or chunking experiments can be tried on a branch or from a checkout and abandoned without touching the stable trunk (Chapter 7).
- **Traceable defects.** Tickets record problems such as upload errors and poor retrieval so that fixes can be tied to check-ins (Chapter 8).
- **Secret hygiene.** Local environment files and credentials are excluded from version control; the notification integration reads credentials from environment variables (Chapters 11 and 12).

5. Fossil Repository Setup and Configuration

5.1 Environments

Fossil was installed on both Windows and Linux. Table 9 records the environment details supplied by the team. The Fossil version in use was 2.28 [52445a27f1], visible in the footer of the pages in Figure 6 and in the dashboard's repository status.

Table 9. Fossil environment on the two platforms

Item	Windows (central host and Niranjan)	Linux (Achyuth)
Fossil executable	C:\Fossil\fossil.exe	/usr/bin/fossil
Project working directory	C:\RAG-Document-QA	Local checkout in the user's home directory (user: achyuth)
Repository file	Central: C:\Fossil\Repos\RAG-Document-QA.fossil Niranjan's clone: C:\Fossil\Repos\RAG-Document-QA-Niran.fossil	Local clone in the home directory
Server address	Port 8090 during the initial collaboration demonstrations; customised UI later at http://localhost:8081	Reaches the central server through the Windows host's network IP address

5.2 Repository Topology

A central repository was created on a Windows laptop and served with Fossil's built-in HTTP server. The host used it locally through localhost, and teammates reached it through the host's network IP address. Each teammate held a clone (a full local copy of the repository) and a checkout (a working directory) of it. This is the common "central hub" arrangement for a small team: every member keeps full history locally, but the hub is the agreed meeting point for synchronisation.

The arrangement has a practical consequence that the team observed: the host's IP address can change after reconnecting to a network, after which the remote URL stored in each clone no longer works and must be updated (the current remote can be inspected with `fossil remote`). This dependency is a trade-off of hosting on a laptop and is discussed in Section 9.3.

5.3 The Default Interface Before Customisation

Figure 5 shows the interface immediately after installation: a generic "Unnamed Fossil Project" header, a top navigation bar with Home, Timeline and Files, and the site-map menu. The menu already lists the built-in modules that the project later relied on: the file browser, project timeline, branches and tags, tickets/bug reports, wiki, repository status, skins and help. Nothing had to be installed to obtain an issue tracker or wiki, which supports the selection rationale in Section 3.4. The same figure is the baseline for the customisation in Chapter 10: the default interface is functional but not specific to DocuMind and does not summarise project state on its front page.

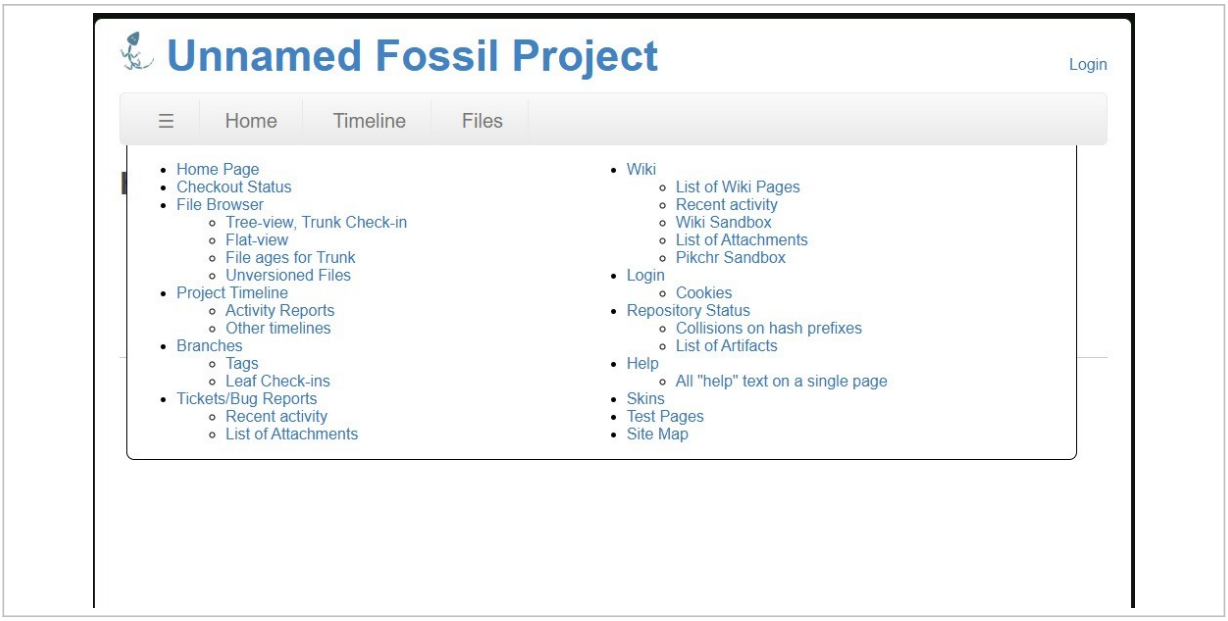


Figure 5. Original Fossil web interface before customisation, showing the default project name and the built-in site-map menu (Fossil 2.28).

5.4 User Accounts and Permissions

Users were created and managed through Admin → Users. Figure 6 shows the user list at the time of capture. The upper table lists the built-in categories and the capability letters each grants (for example, “developer” shows the letters ei, and the Info column explains that it applies to users with the ‘v’ capability); the lower table lists the accounts. Fossil’s capability model is documented in [5].

Table 10. User accounts visible in the user list

Login	Capabilities shown	Interpretation
Niran	s	Setup capability: administrator of the repository
OMEN	v	Developer category: may check in and synchronise changes
achyuth	v	Developer category: granted so that the Linux teammate could push

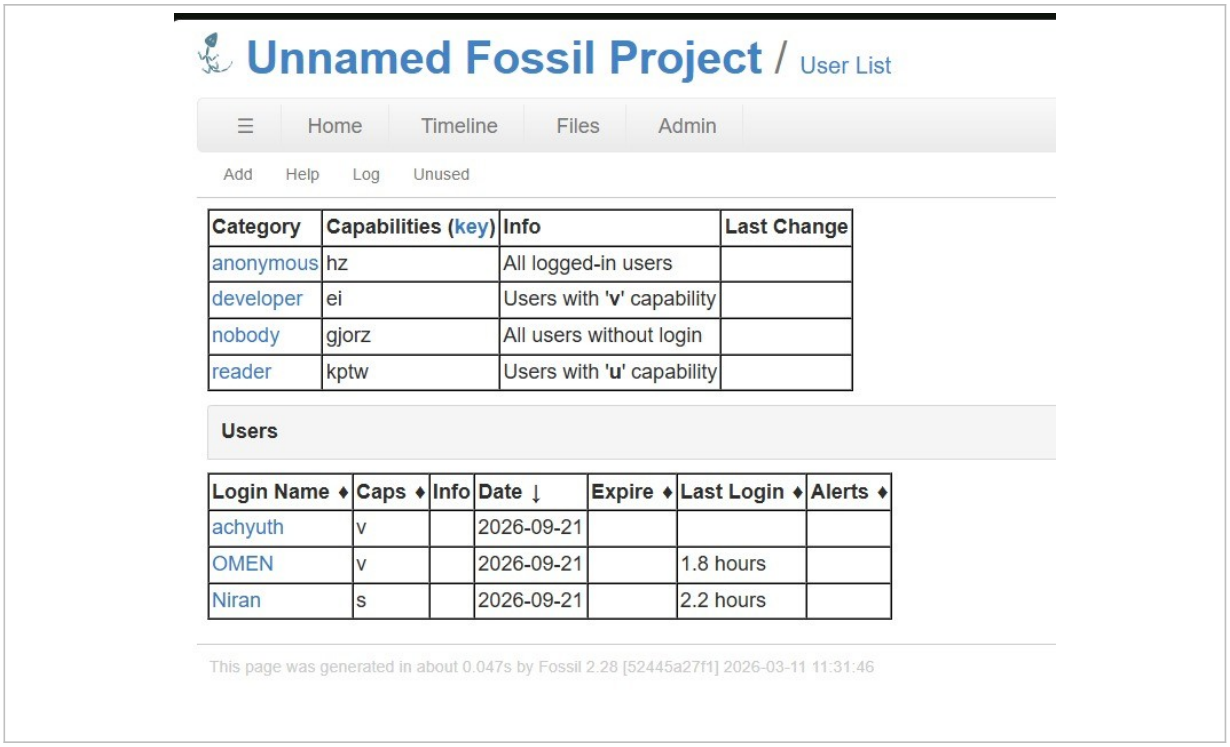


Figure 6. Fossil user list (Admin → Users) showing the capability categories and the accounts Niran, OMEN and achyuth.

The observed sequence is instructive. The Linux teammate could clone the repository and commit locally, because a commit only changes the local repository. Pushing to the central repository, however, requires the check-in capability on the server, which a new account does not have until the administrator grants it. After the account was granted Developer permission the push succeeded and the change appeared on the Windows side (Chapter 9). This shows that Fossil enforces authorisation at the server, not on the client.

It is important to distinguish two layers of protection. Fossil's capabilities control what a user may do through Fossil, whether in the browser or through synchronisation. They do not protect the repository file itself: anyone with operating-system access to C:\Fossil\Repos\RAG-Document-QA.fossil can copy or open it directly, bypassing Fossil's permissions. The repository file and the host machine therefore need their own operating-system-level protection (Chapter 12).

6. Version Control and Change Management

6.1 Distinguishing the Basic Operations

Most confusion among new Fossil users arises from the number of places in which state lives: the working files, the local repository, and the central repository. Table 11 separates the operations the team used according to what each one changes and where.

Table 11. Fossil operations: what each changes and where

Operation	Command	What it does	Changes
Add a file to tracking	`fossil add FILE`	Marks a new file to be included in the next check-in	Checkout state only
Commit (check-in)	`fossil commit -m "message"`	Records the current state of tracked changes as a permanent, uniquely identified version with author, time and message	Local repository (and, with autosync enabled, the remote)
Push	`fossil push`	Sends check-ins and other artifacts from the local repository to the remote (central) repository	Central repository; requires check-in capability
Pull	`fossil pull`	Fetches new artifacts from the remote into the local repository	Local repository only; working files unchanged
Update	`fossil update [VERSION]`	Changes the working files to match a given version (default: latest on the branch); with a branch name, switches branch	Checkout files
Revert	`fossil revert FILE`	Discards uncommitted changes to a file, restoring it to the checked-out version	Checkout files; nothing is removed from history
Check out an older version	`fossil update CHECKIN_ID`	Makes the working files match a historical check-in	Checkout files; later history is untouched
Merge	`fossil merge BRANCH`	Applies the changes of another branch to the current checkout; the result must then be committed	Checkout files; recorded as a merge on commit

Two points deserve emphasis. First, a **commit is not a push**: a commit records history in the local repository, while a push or synchronisation makes it available to others. Fossil can synchronise automatically depending on its `autosync` setting [2], so a separate push is not necessarily required for every commit. In the team's demonstrations the push was performed explicitly, and the Linux push was possible only after permissions were granted; the general principle is that history becomes shared only when it reaches the central repository. Second, **pull does not change the files** a developer is working on. After a pull the developer must run `fossil update` to bring the checkout up to date, which is exactly what Niranjan did to see the Linux teammate's file.

6.2 Inspecting State and Change

Before committing, a developer should understand what is about to be recorded. `fossil status` and `fossil changes` list modified, added and deleted files; `fossil diff` shows the textual differences; `fossil ls` lists tracked files; `fossil info` shows the checked-out version, its hash, parent and the repository it belongs to; and `fossil timeline -n 10` lists the last ten events. Using these commands before every check-in keeps check-ins small and intentional, which is the basis of reviewable history.

6.3 Commit Messages in Practice

A commit message is the main piece of change-management metadata that Fossil keeps for each check-in. The check-in messages visible in the dashboard's recent-activity list illustrate the current practice: "Create new branch named "feature-demo"", "Add feature branch demo file", "Merge feature-demo into trunk", "Test email notification" and "Second notification test". These messages say clearly what was done and are adequate for demonstration check-ins. They do not, however, explain why a change was made, and they carry no reference to a ticket. For the application code the team should adopt a convention such as: a short imperative summary, a blank line, a sentence on the reason for the change, and a reference to the related ticket where one exists. This is a recommendation, not a record of existing practice.

6.4 Timeline and Change History

The timeline is Fossil's principal view of history: it lists check-ins in time order with author, branch, message and hash, and draws the branching and merging relationships. It also presents wiki edits and ticket changes in the same stream, so one page shows everything that happened in the project. The custom dashboard reuses the same data through Fossil's RSS feed (Chapter 10).

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 7 — Repository timeline

Required image: The Fossil Timeline page showing the check-in history, including the feature-demo branch and merge.

Evidence from: Fossil repository (Timeline page)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 7. Fossil timeline showing the initial repository history.

7. Branching, Merging and Rollback

7.1 Branches in Fossil

In Fossil a branch is a named line of development. The name is recorded permanently as a property of the check-ins on that line [7], so the timeline always shows which branch a check-in belonged to, even after the branch has been merged. The default branch is called trunk. Creating a branch does not copy files; it simply makes the next check-in start a new named line.

7.2 The feature-demo Branch and Merge

To demonstrate branching the team created a branch named feature-demo from trunk, added a file FEATURE-DEMO.txt on it, committed and pushed, switched the checkout back to trunk, merged the branch and committed the merge. Table 12 links the steps to the commands and to the check-ins recorded in the repository history. The check-in identifiers are those of the demonstrated repository at the time and are not the latest check-ins today. The times are those shown in the dashboard's recent-activity list.

Table 12. The feature-demo demonstration

Step	Command	Check-in recorded (message, time on 21 Sept 2026)	ID begins
Create branch from trunk	`fossil branch new feature-demo trunk`	Create new branch named "feature-demo", 16:35	b9c4b3bd6b
Add file and commit on branch	`fossil add FEATURE-DEMO.txt` `fossil commit -m "…"`	Add feature branch demo file, 16:44	0c270a140f
Share the branch	`fossil push`	(synchronisation, no new check-in)	—
Return to trunk	`fossil update trunk`	(checkout change only)	—
Merge into trunk	`fossil merge feature-demo`	(files changed in checkout; not yet recorded)	—
Record the merge	`fossil commit -m "Merge feature-demo into trunk"`	Merge feature-demo into trunk, 17:00	a6a1d95729

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 8 — Branch creation and feature check-in

Required image: The timeline (or check-in detail page) showing the feature-demo branch creation check-in and the "Add feature branch demo file" check-in.

Evidence from: Fossil repository (Timeline / check-in page)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 8. Creation of the feature-demo branch and its feature check-in.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 9 — Merge into trunk

Required image: The timeline showing the "Merge feature-demo into trunk" check-in with the branch line rejoining trunk.

Evidence from: Fossil repository (Timeline page)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 9. Merge of feature-demo into trunk, preserved in the repository history.

The two-step nature of a Fossil merge is the main behavioural point: `fossil merge` changes only the working files, and the merge becomes part of history only when the developer commits. The resulting merge check-in has two parents, the previous trunk tip and the tip of the merged branch, which is why the timeline can draw the lines rejoining. The developer also has a chance to build and test the merged result before recording it, which is a safeguard worth using for a multi-component application. The demonstration added a new file, so no textual conflict occurred; the conflict-resolution workflow was therefore not exercised and remains to be practised.

Planned versus demonstrated. Review 1 proposed using separate branches for features and modules and merging them after testing. What has been verified is one real branch and merge cycle. The report does not claim that every module of DocuMind was developed on its own branch.

7.3 Rollback and Recovery

Fossil never edits history in the way that "resetting" a repository does; recovery is done by moving the checkout or by recording new check-ins. Table 13 lists the available techniques and their effect on history.

Table 13. Rollback and recovery techniques

Situation	Technique	Effect on history
Uncommitted edit to one file should be discarded	`fossil revert FILE`	None; the change was never recorded
An earlier version must be inspected or run	`fossil update CHECKIN_ID`	None; later check-ins remain; the checkout shows the old files. Return with `fossil update trunk`
The last update, merge or revert was a mistake	`fossil undo` (documented command for undoing such checkout operations)	None
A committed change must be reversed	Record a new check-in that backs out the change (for example `fossil merge --backout VERSION`) and commit it	A new check-in is added; the original remains visible

Checking out an old version with `fossil update CHECKIN_ID` is a read-mostly operation on the working directory: it does not delete the later check-ins. If a developer edits and commits from such a historical checkout, the new check-in begins a fork from the old version, so the developer should return to the branch tip first. Because history is only ever extended, the project record stays complete, which supports the audit objective of SCM. These commands are documented behaviours; the team's evidence of using them is limited to the command list recorded in Appendix A, and a deliberate rollback exercise would strengthen the report [TEAM: add a screenshot or note if you performed one].

8. Tickets, Tags and Wiki

8.1 Ticket Management

Fossil includes a ticket system whose fields and workflow are defined by a customisable schema, and ticket changes appear in the timeline and trigger alerts. Review 1 planned to track bugs, feature requests and development tasks, for example PDF upload errors, text chunking and retrieval quality. In this report the ticket module is evidenced by the ticket-change email notification that was tested successfully (Chapter 11). The ticket list itself is to be shown in Figure 10.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 10 — Ticket management page

Required image: The Fossil Tickets page showing the tickets created for the project (list view or one ticket's detail).

Evidence from: Fossil repository (Tickets page)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 10. Fossil ticket management used to record project issues.

> *TEAM NOTE (delete before submission): List here (one line each) the tickets that were actually created, with their titles. Do not describe tickets that were not created.*

8.2 Tags and Release Identification

A tag is a label attached to a check-in. Branch names are themselves implemented through tags, and additional tags such as a version number can mark a release. `fossil tag list` shows the tags in the repository, and the Tags page of the web interface shows the same information. The dashboard's Project Version section displays "Version 1.0"; this is text within the skin, not evidence of a Fossil tag. Whether a release tag was created is therefore to be confirmed and evidenced below.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 11 — Tags / release identification

Required image: The Fossil Tags page, or the output of `fossil tag list`, showing the tags present (for example a version tag, if created).

Evidence from: Fossil repository (Tags page or terminal)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 11. Tags used to identify releases and milestones.

8.3 Wiki and Project Documentation

The wiki holds documentation that should live with the code. The repository overview on the dashboard reported six wiki pages at the time of capture (Figure in Section 10.4). The dashboard's Documentation section presents entries for architecture, the RAG pipeline, the technology stack, installation and the project workflow, which correspond to the documentation topics planned in Review 1.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 12 — Fossil wiki and documentation

Required image: The Fossil Wiki page (list of wiki pages, or one documentation page such as the architecture page).

Evidence from: Fossil repository (Wiki page)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 12. Wiki pages used for DocuMind project documentation.

> *TEAM NOTE (delete before submission): Name the six wiki pages that exist. The Documentation cards on the dashboard are links; state which of them point to existing wiki pages.*

9. Cross-Platform Team Collaboration

9.1 Objective

The team included Windows and Linux users, so it was necessary to show that one central repository could serve both. A Windows teammate (account OMEN) created TEAM-DEMO.txt, and the Linux teammate created LINUX-DEMO.txt. Table 14 summarises the verified sequence.

Table 14. Verified cross-platform collaboration sequence

Step	Actor	Action	Result
1	Windows teammate (OMEN)	Created TEAM-DEMO.txt with the text “This file was created by me using Fossil SCM”	File tracked and committed
2	Linux teammate (achyuth)	Created LINUX-DEMO.txt with the text “Linux teammate Fossil test” and committed it locally	Commit succeeded in the local clone
3	Administrator (Niran)	Granted the achyuth account Developer permission on the central repository (Admin → Users)	Account holds the ‘v’ capability
4	Linux teammate	Ran `/usr/bin/fossil push`	Push succeeded
5	Niranjan (Windows)	Pulled and updated the Windows checkout	LINUX-DEMO.txt appeared in the working directory

9.2 Analysis

The demonstration verified the full round trip: a file created on Linux was committed locally, authorised on the server, pushed, pulled on Windows and applied to a working checkout. It also exposed the permission requirement discussed in Section 5.4, which was resolved by account configuration rather than by changing any software. The demonstration used plain text files. Cross-platform issues that matter for real source code, such as line-ending differences between Windows and Linux and path-separator differences in build scripts, were not tested in this exercise and should be watched as the code base grows.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 13 — Windows collaboration evidence

Required image: Windows terminal or Fossil timeline showing TEAM-DEMO.txt being created/committed, and the pull/update that brought in LINUX-DEMO.txt.

Evidence from: Windows teammate (PowerShell or Fossil UI)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 13. Windows side of the cross-platform collaboration.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 14 — Linux collaboration evidence

Required image: Linux terminal showing LINUX-DEMO.txt committed and `/usr/bin/fossil push` succeeding after permissions were granted.

Evidence from: Linux teammate (terminal)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 14. Linux side of the cross-platform collaboration.

9.3 Server Availability

The central repository was hosted on a Windows laptop. Because Fossil is distributed, teammates could keep committing locally while the host was offline, and their work was not lost. Pushing and pulling, however, required the host to be switched on and reachable, and the changing IP address meant the remote URL occasionally had to be updated. This is a real limitation of the laptop-hosted arrangement

and is classified in Chapter 15 as a remaining limitation. A stable always-on host or a secure VPN would remove it (Chapter 18).

10. Custom DocuMind Fossil Dashboard

10.1 Motivation

At the Review 2 demonstration the faculty asked the team to make the Fossil dashboard more interactive. The default home page (Figure 5, Section 5.3) is generic, and the information a team lead needs at a glance, such as the latest check-in, who made it, and how large the repository is, is spread over several Fossil pages. The dashboard was developed by **Niranjan Santhosh** as his contribution to the shared project, and it is presented here as a usability and visibility layer on top of Fossil. Fossil's version-control engine was not modified.

10.2 Design Approach

Fossil's appearance is controlled by a skin, which consists of a header, footer, style sheet, optional JavaScript and a few detail settings, all stored in the repository configuration and editable through the web interface [8]. The dashboard is implemented as a custom skin, currently named draft1, using HTML, CSS and JavaScript only. The visual theme is a professional navy, slate and rose palette with card-based layout. The JavaScript is loaded through Fossil's js.txt mechanism, which lets a skin supply scripts that run in the browser. Because the existing HTML, CSS and JavaScript are stable, further work builds on them rather than rebuilding working sections.

10.3 Implemented Sections

Table 15. Dashboard sections

Section	Content	Nature
System Status	Cards: system, repository, AI/RAG, branch	Presentation cards
Project Information	About DocuMind and the RAG approach	Static text
Quick Actions	Links to timeline, files, branches, tickets and documentation	Navigation
DocuMind Services	Language model, embeddings, vector database, backend	Presentation cards
RAG Workflow	Seven-step flow from upload to answer	Static diagram
Documentation	Architecture, RAG pipeline, technology stack, installation, project workflow	Navigation
Repository Activity	Cards linking to timeline, files, branches, tickets, wiki	Navigation
Repository Overview	Check-ins, files, wiki pages, artifacts, project days, repository size	**Dynamic**
Project Team	The actual Fossil user Niran, shown as Repository Administrator	Based on real account
Recent Activity	Five most recent check-ins with message, author, branch, time	**Dynamic** (timeline RSS)
Project Version	Version 1.0, repository DocuMind, SCM Fossil, RAG architecture	Static text
Repository Status	Latest check-in, author, branch, latest activity, check-in ID, Fossil version	**Dynamic**

10.3.1 Dynamic Content

Three sections are populated at page-load time. **Recent Activity** reads Fossil's timeline RSS endpoint, `/timeline.rss?y=ci&n=5`, which returns the five most recent check-ins as an XML feed. The script parses the feed and renders each entry's message, author, branch and timestamp. Using Fossil's own feed keeps the dashboard consistent with the repository without any separate database. **Repository Overview** and **Repository Status** present live figures derived from the repository's own information pages [TEAM TO CONFIRM which Fossil page(s) the script reads, e.g. the repository statistics page]. During development a CSS grid layout caused misalignment in the status cards; replacing it with a wrapping flexbox layout resolved the problem.

Project Team deliberately shows only the real Fossil account (Niran, Repository Administrator). The team chose not to create placeholder or fictitious members, so the dashboard reflects actual repository users.

10.4 Evidence and Analysis

Figure 15 shows the header, navigation bar, System Status and the start of Project Information. The navigation retains Fossil's standard menu (Home, Timeline, Files, Branches, Tags, Forum, Chat, Tickets, Wiki, Admin, Logout), so the customisation adds a front page without removing any native function. Figure 16 shows the Quick Actions cards, each of which maps to a native Fossil page.

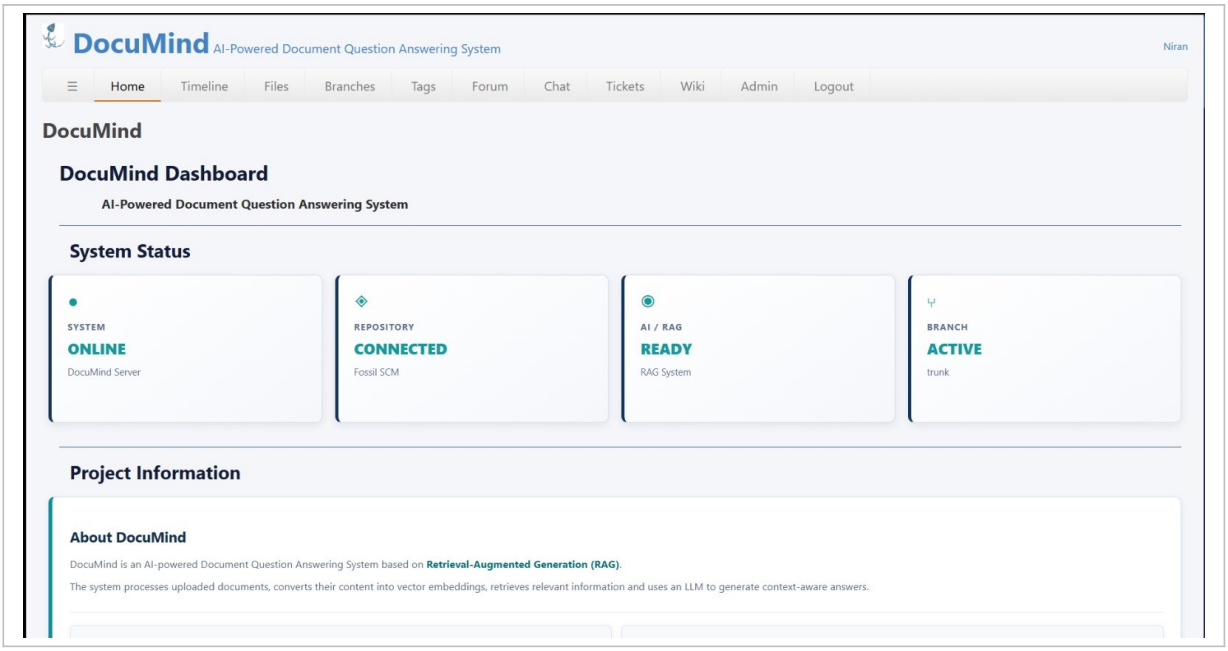


Figure 15. Customised DocuMind Fossil dashboard showing the navigation bar, System Status and Project Information sections.

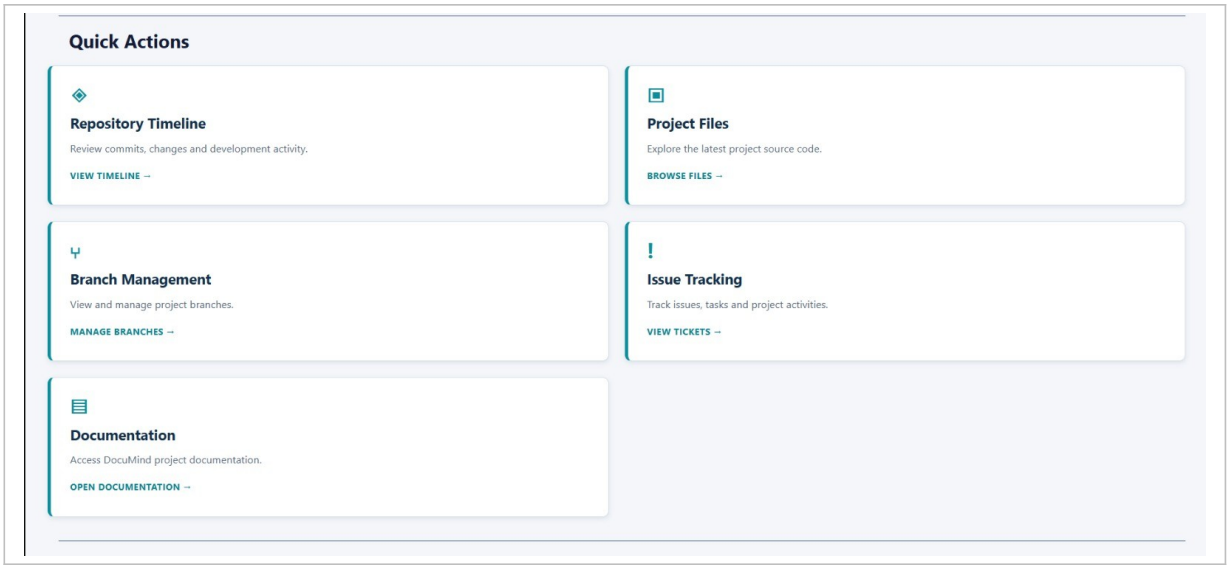


Figure 16. Quick Actions section linking to the timeline, files, branches, tickets and documentation.

Figure 17 is to show the DocuMind Services cards and the RAG Workflow strip, which presents the flow of Section 4.5 as seven steps.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 17 — DocuMind services and RAG workflow

Required image: The DocuMind Services and RAG Workflow sections of the dashboard, after the vector-store label has been corrected to PostgreSQL/pgvector.

Evidence from: Fossil repository (custom skin, Niranjan Santhosh)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 17. DocuMind Services and RAG Workflow sections of the custom dashboard.

> *TEAM NOTE (delete before submission): The screenshot supplied earlier shows "Qdrant" in both the Vector Database card and the Qdrant workflow step. The application uses PostgreSQL with pgvector. Edit the skin text, re-capture this figure, and keep this placeholder until then.*

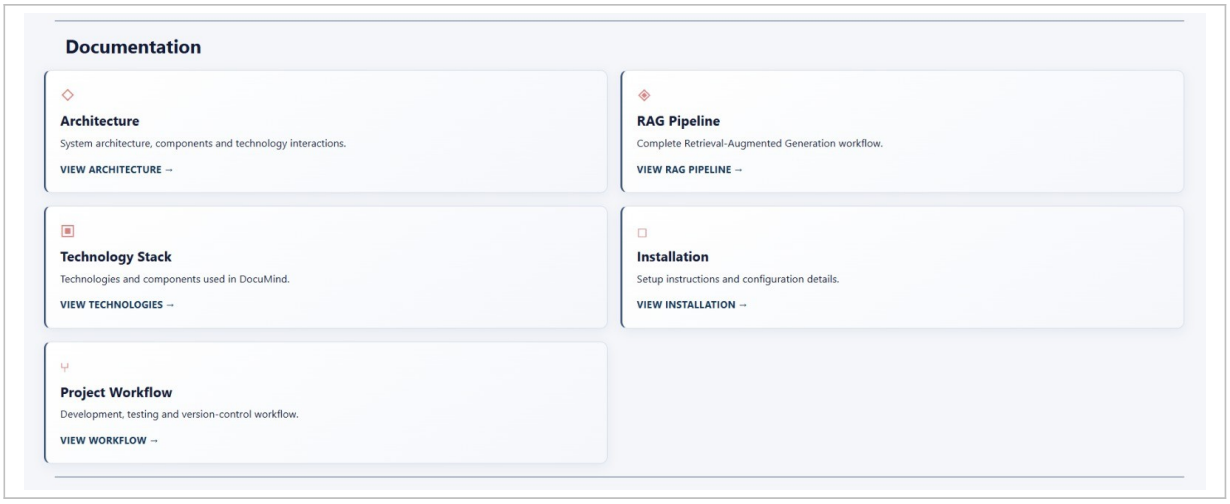


Figure 18. Documentation section of the dashboard.

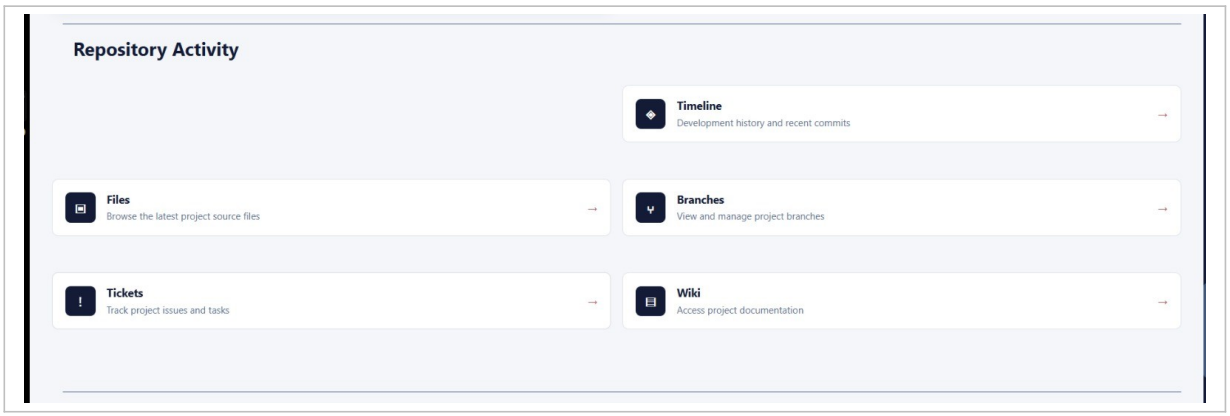


Figure 19. Repository Activity navigation cards (timeline, files, branches, tickets, wiki).

Figure 20 shows the dynamic Repository Overview and Repository Status. At the time of capture the overview reported 11 check-ins, 163 files, 6 wiki pages, 219 artifacts, a project age of 29 days and a repository size of 1.6 MB; the status panel reported the latest check-in as “Second notification test” by Niran on trunk at 18 Oct 2026, 19:33, with Fossil version 2.28 [52445a27f1]. These are a snapshot, not final figures. The check-in identifier is a 64-character hexadecimal hash, shown as a link. The value that stands out analytically is that the status panel’s latest check-in matches the top entry of the Recent Activity list in Figure 21, which shows that two independently rendered sections agree with the repository.

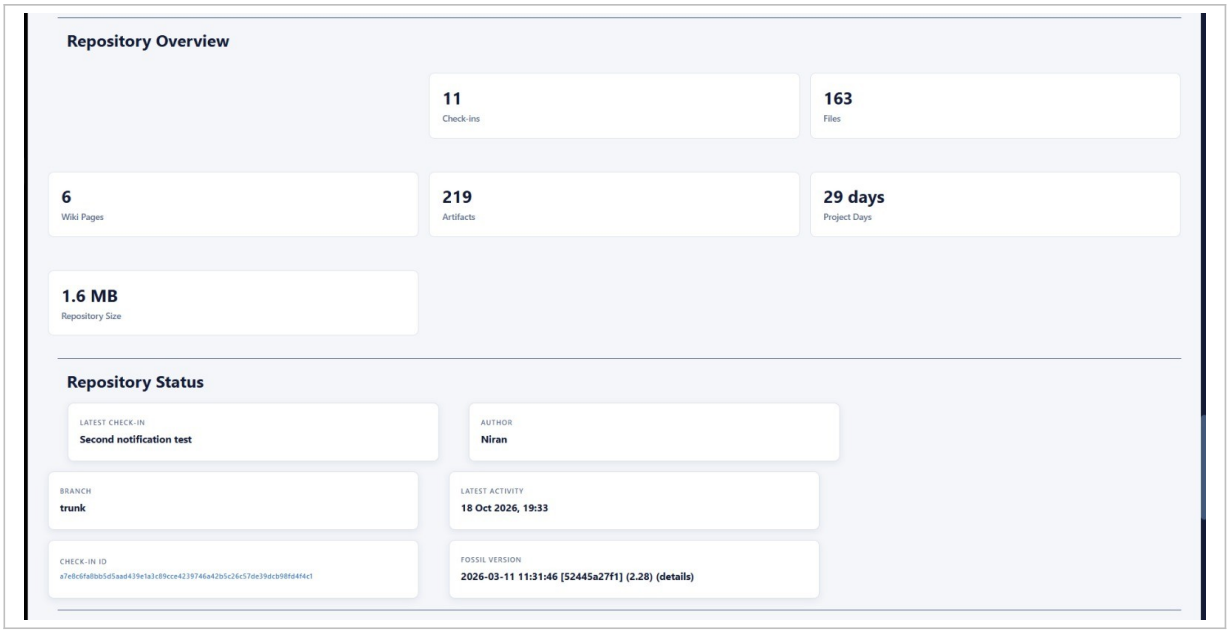


Figure 20. Dynamic Repository Overview and Repository Status sections.

Figure 21 shows the Project Team entry, the five most recent check-ins and the Project Version block. The recent activity list also documents the history discussed elsewhere: the creation of the feature-demo branch (21 Sept 2026, 16:35), the feature file (16:44), the merge into trunk (17:00), and the two notification-test check-ins on 18 Oct 2026 (19:09 and 19:33).

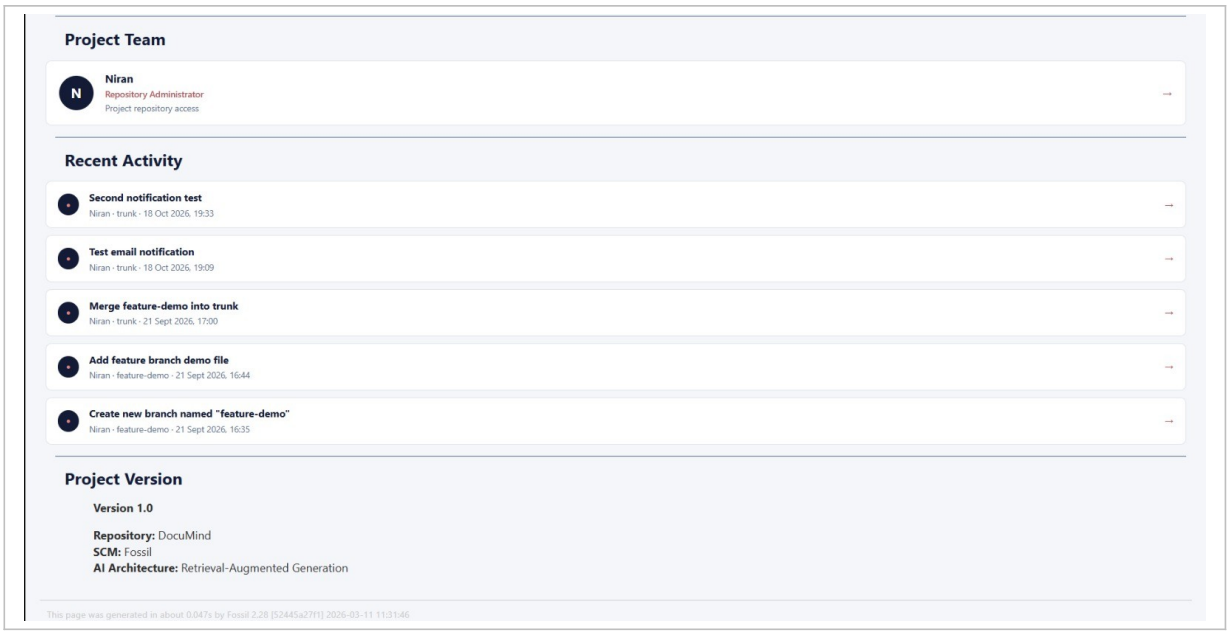


Figure 21. Project Team, dynamic Recent Activity and Project Version sections.

10.5 Limitations of the Dashboard

- Only the Recent Activity, Repository Overview and Repository Status sections are described as dynamic. System Status badges, DocuMind Services badges and the Project Version block are presentation elements unless they are later connected to real health checks or tags; they should not be read as live monitoring of the application.
- The dashboard does not change what Fossil can do; it improves how information is presented. Access control still follows Fossil capabilities.

- The dashboard depends on the Fossil skin and on browser-side JavaScript; it would need re-testing after a Fossil upgrade.

11. Fossil Email Notification Integration

11.1 Requirement

The second item of faculty feedback was that the administrator should be notified of repository activity. Without notifications the administrator must repeatedly open the web interface to see whether anything changed. The notification integration was configured by **Harisankar C Mooss**, with the team's collaboration, as his principal contribution.

11.2 Fossil Alerts and the Chosen Architecture

Fossil has a built-in notification system: users subscribe to categories of events, Fossil composes the alert messages, and an outgoing-mail method delivers them [4]. Fossil does not itself speak to Gmail, so the team used the "pipe" send method, in which Fossil passes each outgoing message to an external command. That command is a Python script that sends the message through Gmail's SMTP service. The resulting chain is shown in Table 16.

Table 16. Notification chain

Stage	Component	Role
1	Fossil SCM	Detects events (check-in, ticket change, and so on) and records them for subscribers
2	Fossil alert generation and processing	Composes the alert email for each subscriber
3	Python email sender (C:\Fossil\ fossil_email_sender.py)	Receives the message on standard input and sends it
4	Gmail SMTP	Delivers the message over the Python standard library's SMTP client [18]
5	Gmail inbox	The administrator receives the alert

This is an example of extending Fossil through an external tool rather than modifying it: the integration needs no change to Fossil's source and can be replaced with another mail relay by changing the script.

11.3 Configuration

Table 17. Fossil email settings (non-secret values only)

Setting	Value	Purpose
email-admin	The administrator's Gmail address (withheld)	Administrator address for alert-system messages
email-self	The configured sender/administrator address (withheld)	"From" address of outgoing alerts
email-listid	documind	Identifies the alert stream
email-subname	DocuMind	Subject-line prefix
email-url	The local Fossil UI URL	Base URL placed in links inside alerts
email-send-method	pipe	Hand outgoing messages to an external command

Setting	Value	Purpose
Pipe command	C:\Python314\python.exe C:\Fossil\ fossil_email_sender.py	The program that receives each message

Credentials are not stored in the script or in the repository. The script reads the sender account and a Gmail App Password from two environment variables, FOSSIL_GMAIL_USER and FOSSIL_GMAIL_APP_PASSWORD, which were configured privately on the host. An App Password is a separate, revocable credential that Google issues for an application when the account uses two-step verification [19], so the main account password is never used. The values of these variables are intentionally not reproduced in this report.

One consequence of the configuration deserves mention. Because email-url is the local Fossil UI address, links inside alerts resolve correctly only on the host machine. Recipients on other machines would need an address that they can reach, which depends on the hosting changes in Chapter 18.

11.4 Testing and Verification

The path was tested in stages. First the Python script and Gmail delivery were tested without Fossil, by piping a prepared message file into the script: `type C:\Fossil\test_email.eml | python C:\Fossil\fossil_email_sender.py`. The message arrived in Gmail. Next, Fossil itself was tested. The timeline records two check-ins made on 18 October 2026 for this purpose (“Test email notification” at 19:09 and “Second notification test” at 19:33). Table 18 gives the status of each notification type.

Table 18. Notification types and their status

Notification type	Status	Notes
Pipe test message (.eml)	Tested	Arrived in Gmail
Check-ins	Tested successfully	Alert received by the administrator
Ticket changes	Tested successfully	Alert received by the administrator
Announcements	Configured	Subscription enabled for the administrator
Wiki updates	Configured	Subscription enabled for the administrator
Forum-related notifications	Configured	Subscription enabled for the administrator
User-permission changes	Configured	Subscription enabled for the administrator

The administrator’s email subscriber was verified. Subscriptions for the other team members were being expanded; this report does not claim that every teammate is subscribed.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 22 — Fossil email-server settings

Required image: Fossil Admin → Email-Server (or Settings) page showing email-send-method pipe, the pipe command, email-listid and email-subname. Hide or crop the administrator's address if you prefer; never show passwords.

Evidence from: Notification setup (Harisankar C Mooss)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 22. Fossil email settings used for the Python pipe integration (no secrets shown).

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 23 — Test email in Gmail

Required image: The Gmail inbox showing the message sent with the `type … | python …` test command, and optionally the terminal command.

Evidence from: Notification setup (Harisankar C Mooss)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 23. Test message sent through the Python script and Gmail SMTP and received in the inbox.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 24 — Check-in notification email

Required image: The actual Fossil check-in alert email received in Gmail.

Evidence from: Notification setup (Gmail inbox)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 24. Fossil check-in notification received by the administrator.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 25 — Ticket notification email

Required image: The actual Fossil ticket-change alert email received in Gmail.

Evidence from: Notification setup (Gmail inbox)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 25. Fossil ticket-change notification received by the administrator.

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 26 — Administrator subscriber configuration

Required image: The Fossil subscriber/alert settings page showing the administrator's subscription topics (announcements, check-ins, tickets, forum, wiki, user permissions).

Evidence from: Fossil repository (Alerts / Subscribers page)

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 26. Administrator subscriber and topic configuration.

11.5 Automatic Background Processing

In the first implementation, delivery required running `fossil alerts send` manually: Fossil queued the alerts but did not send them until the command was run. The team investigated the `email-autoexec` setting and Fossil's backoffice mechanism, which is the part of Fossil that performs housekeeping work such as processing pending alerts in the background [4]. Delivery immediately on ticket creation was not working at that stage. The team has since confirmed that automatic notification processing is working and that all planned notification types are complete.

> *TEAM NOTE (delete before submission): Add one or two sentences on how automatic processing was achieved (for example the exact setting or scheduled task used) and attach evidence if available. The report currently states only the confirmed outcome.*

Two layers should be distinguished when assessing this work. Subscriber and topic configuration decides who is notified about what, and it was completed for the administrator. Background delivery decides when the messages are actually sent, and the team reports that it now works without manual commands. Because delivery depends on the Fossil server process, the pipe command and the environment variables all being available on the host, it is advisable to re-test after the host is restarted.

12. Configuration Management, Security and Backup

12.1 What Is Versioned and What Is Not

Configuration management here means keeping every file needed to build and run DocuMind under version control, so that any version can be reproduced, while keeping environment-specific data and secrets outside the history. Table 19 states the intended policy; the first column is a recommendation unless marked otherwise.

Table 19. Configuration items and handling policy

Item	Under version control?	Reason
Backend source and Maven build file	Yes	Required to build
Frontend source and dependency list	Yes	Required to build
Flyway migration files	Yes	Schema must match code version
Docker/Docker Compose configuration	Yes	Reproducible services
Model names (llama3.1:8b, nomic-embed-text) in configuration	Yes	A model change is a tracked change
Local environment files and credentials	No	Secrets must not enter immutable history
Gmail credentials	No (environment variables)	Only variable names appear in documentation
Build outputs and installed dependencies	No	Regenerable

Fossil can exclude files from commands such as `fossil add` and `fossil extras` using an ignore list, which can itself be versioned [TEAM TO CONFIRM that an ignore list was configured]. The point is especially important in Fossil because history is permanent: once a secret is committed, it stays in the repository's artifacts and in every clone. Fossil does have a way to remove artifacts ("shunning"), but it is a deliberate administrator action and cannot be relied on to un-leak a credential, so prevention is the correct control. The notification integration followed this principle by using environment variables.

12.2 Access Control and Transport Security

- **Fossil capabilities** limit what each account can do through Fossil (Section 5.4).
- **Operating-system permissions** must protect the repository file and the machine hosting it, because direct file access bypasses Fossil's capabilities.
- **Network exposure:** the server was reached over HTTP on a local network. Plain HTTP does not encrypt traffic, so for use beyond a trusted network the team should use HTTPS or a VPN (recommendation, Chapter 18).

12.3 Backup and Recovery

Fossil's distributed design means every clone holds a copy of the project history, which gives informal redundancy. Fossil's documentation cautions that a clone is not equivalent to a full backup, because some repository-specific configuration, such as user accounts, is not copied by a clone, and recommends dedicated backup methods rather than copying a repository file that may be in use [6]. The recommended procedure for this project is: (1) take scheduled backups of C:\Fossil\Repos\RAG-Document-QA.fossil using a Fossil-supported method; (2) store copies off the host; (3) periodically restore a backup to a separate location and open it to confirm it works. A backup that has never been

restored is untested. **No backup procedure was implemented or evidenced during this project**; this section is therefore a recommendation and is classified as future work in Chapter 15.

13. Review History and Faculty Feedback

13.1 Review 1 and Review 2

Review 1 was a written document. It introduced Fossil SCM, explained the problem, and described the planned use of version control, branches, change management, tickets, release tags, wiki and configuration management. **Review 2** was not a written report; it was a practical demonstration in which the team explored Fossil and showed the features implemented so far. The faculty gave two pieces of feedback: make the dashboard more interactive, and add notifications for the administrator.

13.2 Plan Versus Evidence

Table 20. Review 1 plan compared with the evidence in this report

Planned practice	Status	Evidence
Version control (commit, compare, restore)	Demonstrated	Timeline, check-ins, command list (Chapter 6)
Branch management for features and modules	Partly demonstrated	One real branch and merge (feature-demo). Per-module branching not verified
Change management with meaningful messages	Demonstrated, with room to improve	Messages describe the action but seldom the reason (Section 6.3)
Ticket management	Configured and notification-tested	Ticket-change alert received; ticket list to be shown
Release tags	To be confirmed	Dashboard shows Version 1.0 text; tag list to be shown
Wiki documentation	Demonstrated	6 wiki pages in the repository snapshot
Configuration management and secret protection	Partly demonstrated	Secrets kept in environment variables in the notification integration; ignore-list evidence to be added

13.3 Feedback and Response

Table 21. Faculty feedback and the team’s response

Feedback	Response	Owner	Chapter
Make the dashboard more interactive	Custom skin with navigation cards and dynamic overview, recent activity and status	Niranjan Santhosh	10
Add notifications for the administrator	Fossil alerts piped to Python and Gmail SMTP; check-in and ticket alerts tested; automatic processing working	Harisankar C Mooss	11

14. Comparison Study

14.1 Comparing Different Categories Fairly

Fossil, Git, GitHub and GitLab are not four versions of the same thing, and a fair comparison must compare like with like. **Fossil** is a distributed version-control system that also contains project-management tools and a web interface [1]. **Git** is a distributed version-control system only; it manages history and leaves issue tracking, wikis, hosting and notifications to other tools [9]. **GitHub** and **GitLab** are collaboration and hosting platforms built around Git that add issues, wikis, review workflows, notifications and automation [10]. The meaningful comparison for a project like DocuMind is therefore between Fossil as an integrated tool and the combination of Git with a hosting platform.

The comparison below is based on official documentation [2][9][10] and on our own use of Fossil. The team did not use Git, GitHub or GitLab on this project, so no measured comparison is claimed.

14.2 Characteristics Compared

Table 22. Comparison of Fossil, Git, GitHub and GitLab

Characteristic	Fossil	Git	GitHub	GitLab
Category	DVCS with integrated project tools	DVCS (tool only)	Hosting/collaboration platform on Git	Hosting/DevOps platform on Git
Version control	Own engine; repository is one SQLite file	Own engine; object database in a directory	Uses Git	Uses Git
Branching and merging	Named, permanent branches; merge applied to checkout then committed	Lightweight movable branches; merge and rebase	Git plus pull requests and review	Git plus merge requests and review
Offline work	Local commits; sync later (autosync can be disabled)	Fully offline	Git offline; web features need a connection	Git offline; web features need a connection
Web interface	Built in	None built in (read-only browsers only)	Rich	Rich
Tickets/issues	Built in, customisable fields	None	Issues	Issues, boards
Wiki/ documentation	Built in	None	Wikis	Wikis
Notifications	Built-in email alerts; needs mail delivery set up (our Python/Gmail pipe)	None (hooks possible)	Built in	Built in
Access control	Per-user capability letters	Depends on host or filesystem	Roles and branch protection	Roles and protected branches
Hosting	Self-host with one executable; some third-party hosting	Any Git server or filesystem	Cloud (and Enterprise Server)	Cloud or self-managed
Automation and integrations	Hooks and external scripts; no built-in CI service	Hooks; large ecosystem	Actions, Apps, API	Built-in CI/CD, API

Characteristic	Fossil	Git	GitHub	GitLab
Setup effort	Low	Low	Low (account)	Low in cloud; higher self-managed
Ecosystem and tool support	Smaller	Very large	Very large	Large

14.3 Fit for the DocuMind Project

For this project's needs, Fossil's fit is strong where the project values self-containment: one executable and one repository file provide version control, tickets and wiki with no external account, which suited a three-person course project and the study objective. Its fit is weaker where a hosting platform is stronger: there is no built-in continuous-integration service to build the Spring Boot and React code on every push, no pull-request review workflow comparable to GitHub or GitLab, and the server was only as available as the team's laptop. GitHub or GitLab would also have removed the host-availability problem and the changing-IP problem, at the cost of depending on an external service and using separate tools for some functions. Git-based tools have a far larger ecosystem of editors, integrations and community knowledge, and a team planning a long-lived production product might reasonably choose them.

The trade-offs are acceptable for this project because the team is small, the work is a time-bounded course project, the missing features (CI, review workflow) were not requirements, and the main practical weaknesses were reduced by the customisations in Chapters 10 and 11. If the requirements grew, Fossil can export its history to Git (the `fossil git export` command, documented in [1]), so the choice is not irreversible. Fossil was therefore a justified selection, and it is reasonable to say that it was suitable for this project, not that it is superior in general.

15. Fossil Advantages, Limitations and Practical Mitigation

15.1 Observed Advantages

- **Integration.** Version control, tickets, wiki, timeline and user administration come from one program, so the team needed no additional services for project management.
- **Simple deployment.** A single executable and a single repository file made installation on Windows and Linux straightforward and made the repository easy to copy.
- **Complete local history.** Every clone holds the project history, so work continued when the server was unreachable.
- **Extensibility without modification.** The skin system and the pipe mail method allowed the dashboard and the Gmail integration without changing Fossil.
- **Unified activity view.** The timeline combines check-ins, wiki edits and ticket changes, which supports status accounting.

15.2 Limitations and What Was Done About Them

A list of disadvantages alone is not useful. Table 23 records, for each limitation encountered, the action taken, the evidence, and a classification: **overcome and tested**, **improved but not eliminated**, or **remaining limitation / future work**. The classification is conservative: a limitation is called overcome only where the report contains evidence.

Table 23. Limitations, actions taken and classification

Limitation	Action taken	Evidence	Classification
Default dashboard lacks a team-specific visual overview	Custom DocuMind skin with navigation cards and dynamic sections	Dashboard figures (Chapter 10)	Overcome and tested
Project information spread over several Fossil pages	Repository overview, recent activity and status surfaced on the front page from Fossil's own data	Dashboard figures; status matches timeline	Improved but not eliminated (some status badges are static)
Administrator must watch the web interface for changes	Fossil alerts → Python → Gmail SMTP; check-in and ticket alerts tested; other topics configured; automatic processing confirmed	Emails (to be inserted); Chapter 11	Overcome and tested for the administrator; team-member subscriptions still being expanded
Teammates cannot push without permission	Separate accounts with Developer capability	User list; Linux push demonstrated	Overcome and tested
Server address changes (laptop host); server must be on to sync	Local commits continue offline; remote URL updated when IP changes	Observed during collaboration	Remaining limitation (needs always-on host or VPN)
Branching and merging are unfamiliar to beginners	Practised on feature-demo: branch, commit, update, merge, commit	Branch and merge check-ins	Improved but not eliminated (no conflict exercise)
Risk of confusing checkout and history during rollback	Documented distinction between revert, update and merge; commands practised	Chapter 7, Appendix A	Improved but not eliminated (no recorded recovery drill)
Repository loss	Recommended scheduled, tested backups (Section 12.3)	None; not implemented	Remaining limitation / future work

Limitation	Action taken	Evidence	Classification
Smaller integration ecosystem; no built-in CI	Extended Fossil through Python and Gmail; no CI service added	Chapter 11	Improved but not eliminated
Notification links point to a local URL	email-url set to the local UI address; works on host only	Configuration	Remaining limitation (needs reachable address)
Unencrypted HTTP on local network	Not addressed; HTTPS/VPN recommended	—	Remaining limitation / future work

15.3 Summary of Unresolved Items

The limitations that remain are those that depend on infrastructure rather than on Fossil's own features: host availability, a reachable URL for notification links, encrypted transport, scheduled and tested backups, and a continuous-integration service. These are proposed as future work in Chapter 18. The report does not claim that all disadvantages of Fossil were eliminated.

16. Testing and Results

16.1 Verification of the SCM Implementation

The SCM functions were verified by direct use and by cross-checking independent views of the same repository. Table 24 states what was checked and the evidence. “Observed” means the result is visible in a supplied figure or was reported by the team; no performance measurements are given.

Table 24. SCM verification summary

Check	Expected result	Observed result	Evidence
Windows–Linux synchronisation	Linux commit reaches Windows checkout	LINUX-DEMO.txt appeared after pull and update	Figure 13, Figure 14
Permission enforcement	Push refused until Developer capability granted; succeeds afterwards	Push succeeded after permission change	Figure 6
Branch and merge	Branch appears in history; merge check-in joins it to trunk	feature-demo creation, feature commit and merge check-in present	Check-ins on 21 Sept 2026 (Figure 21)
Dashboard data consistency	Latest check-in in status equals newest entry in recent activity	Both show “Second notification test”, Niran, trunk, 18 Oct 2026 19:33	Figure 20, Figure 21
Pipe and Gmail delivery	Test message arrives in Gmail	Arrived	Test email (to be inserted)
Check-in notification	Alert on new check-in	Received	Email (to be inserted)
Ticket notification	Alert on ticket change	Received	Email (to be inserted)
Automatic notification delivery	Alerts sent without manual command	Confirmed working by the team	Team confirmation; mechanism to be added

16.2 Testing of the DocuMind Application

The RAG application is confirmed by the team to be fully working: documents can be uploaded and processed, and questions return answers generated from the document content. The team has not collected quantitative measurements, so this report gives no accuracy, retrieval-quality or latency figures. Table 25 is left for the team to complete if measurements are made; otherwise the rows should be deleted.

Table 25. Measurements not yet collected

Measure	Value	Conditions
Document ingestion time (per document)	[Not measured]	[document size, hardware]
Question-to-answer response time	[Not measured]	[model, hardware, GPU/CPU]
Retrieval relevance (e.g. share of questions whose answer is in the retrieved chunks)	[Not measured]	[question set]

A qualitative test plan the team can record results against is given in Table 26. The result column is intentionally blank; entries must come from actual runs.

Table 26. Functional test cases for DocuMind (results to be recorded by the team)

Test case	Expected behaviour	Result
Upload a supported document	Document accepted, processed and available for questions	[to be recorded]
Ask a question answered in the document	Answer reflects document content	[to be recorded]
Ask a question not covered by the documents	Answer does not present unsupported content as fact	[to be recorded]
Ask after uploading several documents	Relevant document is used	[to be recorded]
Restart services and ask again	Stored documents still searchable	[to be recorded]

[SCREENSHOT PLACEHOLDER — INSERT IMAGE HERE]

Figure 27 — Testing and result evidence

Required image: Screenshot(s) of test results, e.g. an answered question with its source, a passing build/test run, or the filled test-case table.

Evidence from: DocuMind application / terminal

(Click inside this box, delete this text, then Insert → Pictures. Keep the caption below.)

Figure 27. Testing evidence for the DocuMind application.

16.3 Discussion

The results show that Fossil can support the full life of a small multi-component project: history, branching, permissions, documentation and notifications were all exercised. Three observations are most significant. First, the most valuable customisations (dashboard and notifications) were built from Fossil's own extension points (skins, the RSS feed, the pipe mail method), which shows that Fossil's limited ecosystem can be offset by its openness to scripting. Second, the problems that actually arose were mostly operational (permissions, server address, delivery processing) rather than defects in version control, so the principal risks are in hosting and administration. Third, the evidence for some planned practices is thinner than for others: per-module branching, release tags, backup and recovery drills are the main gaps, and the report labels them as such.

17. Team Contributions

All three members contributed collaboratively to the DocuMind RAG application. Table 27 lists the principal documented contributions to the SCM and customisation work. It is not an exhaustive division of every task.

Table 27. Principal documented contributions

Member	Registration no.	Principal documented contribution
Harisankar C Mooss	24MIS0248	Collaborative development of the RAG application; configured the Fossil email notification integration using Python and Gmail SMTP
Achyuth P N	24MIS0249	Collaborative development of the RAG application; participated in the SCM demonstrations, including Linux-based repository use and pushing a change to the central repository
Niranjan Santhosh	24MIS0450	Collaborative development of the RAG application; developed the custom interactive DocuMind Fossil dashboard

18. Future Enhancements

- **Always-on hosting or secure access.** Host the central repository on a stable machine or a server, or reach it through a VPN, and serve it over HTTPS. This removes the dependence on a laptop and a changing IP address.
- **Reachable notification links.** Set the email URL to an address recipients can reach, and extend subscriptions to all team members.
- **Scheduled, tested backups.** Automate backups of the repository file, store copies off the host, and rehearse a restore.
- **Conflict and rollback exercises.** Practise a deliberate merge conflict and a recovery using `fossil update` and a back-out merge, and record the results.
- **Release tags.** Tag stable milestones (for example, a tested version of the RAG pipeline) and show them on the dashboard from the real tag list.
- **Live service status.** Connect the dashboard's status badges to real health checks of the backend, database and Ollama, and correct the vector-store label.
- **Commit and ticket conventions.** Adopt a message convention that states the reason for a change and links the ticket.
- **Build automation.** Add an external build/test runner triggered by repository changes, since Fossil has no built-in CI service.
- **Measured evaluation of RAG.** Measure retrieval relevance, response time and answer faithfulness on a fixed question set.

19. Conclusion

This project built DocuMind, a working retrieval-augmented document question-answering system on Spring Boot, React/TypeScript, PostgreSQL with pgvector and Ollama-hosted models, and managed it with Fossil SCM. The work went beyond installing a version-control tool. The team set up a central repository, administered user accounts and permissions, demonstrated collaboration between Windows and Linux, and carried out a real branch and merge cycle. In response to faculty feedback, Niranjan Santhosh built a custom, partly dynamic DocuMind dashboard within Fossil's skin system, and Harisankar C Mooss integrated Fossil alerts with Python and Gmail SMTP so that the administrator is notified of check-ins and ticket changes, with automatic processing now confirmed.

The report also assessed Fossil critically. Its integrated design suited a small team and a course focused on SCM, while its smaller ecosystem, lack of built-in continuous integration and dependence on the team's own hosting are real costs. Several limitations were overcome and tested, others were improved, and a few remain: host availability, encrypted transport, notification link reachability and tested backups. Compared fairly with Git and the Git-based platforms GitHub and GitLab, Fossil was a suitable and justified choice for this project, though not a universal one.

References

> *TEAM NOTE (delete before submission): Re-open each URL, confirm it still resolves, and add an "accessed" date before submission. Cite only what you actually used.*

[1] Fossil SCM. "Fossil: Home" and documentation index. https://fossil-scm.org/home/doc/trunk/www/index.wiki

[2] Fossil SCM. "Fossil Versus Git." https://fossil-scm.org/home/doc/trunk/www/fossil-v-git.wiki

[3] Fossil SCM. "Fossil Concepts" (artifacts, check-ins, hashes). https://fossil-scm.org/home/doc/trunk/www/concepts.wiki

[4] Fossil SCM. "Email Alerts and Notifications." https://fossil-scm.org/home/doc/trunk/www/alerts.md

[5] Fossil SCM. "Administering User Capabilities." https://fossil-scm.org/home/doc/trunk/www/caps/

[6] Fossil SCM. "Backing Up a Remote Fossil Repository." https://fossil-scm.org/home/doc/trunk/www/backup.md

[7] Fossil SCM. "Branching, Forking, and Tagging." https://fossil-scm.org/home/doc/trunk/www/branching.wiki

[8] Fossil SCM. "Theming" (skins). https://fossil-scm.org/home/doc/trunk/www/customskin.md

[9] S. Chacon and B. Straub. Pro Git. https://git-scm.com/book

[10] GitHub Docs, https://docs.github.com ; GitLab Docs, https://docs.gitlab.com

[11] P. Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems 33, 2020. arXiv:2005.11401

[12] pgvector: open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector

[13] PostgreSQL Documentation. https://www.postgresql.org/docs/

[14] Spring Boot. https://spring.io/projects/spring-boot

[15] Ollama. https://ollama.com ; model library entries for llama3.1 and nomic-embed-text. https://ollama.com/library

[16] Flyway Documentation. https://documentation.red-gate.com/flyway

[17] Docker Compose Documentation. https://docs.docker.com/compose/

[18] Python Software Foundation. "smtplib — SMTP protocol client." https://docs.python.org/3/library/smtplib.html

[19] Google Account Help. "Sign in with app passwords." https://support.google.com/accounts/answer/185833

[20] P. Bourque and R. Fairley (eds.). Guide to the Software Engineering Body of Knowledge (SWEBOK), Version 3.0. IEEE Computer Society, 2014.

[21] React documentation, https://react.dev ; Vite documentation, https://vite.dev

Appendix A: Selected Fossil Commands

Commands recorded by the team during the project are listed with their purpose. The Linux equivalents use `/usr/bin/fossil`, for example `/usr/bin/fossil status`.

Table 28. Commands used

Command	Purpose
`fossil version`	Show the installed Fossil version (2.28 [52445a27f1])
`fossil status`	Show the checkout version and uncommitted changes
`fossil changes`	List files changed in the checkout
`fossil ls`	List files tracked in the checkout
`fossil timeline -n 10`	Show the ten most recent timeline events
`fossil info`	Show the current checkout’s version and repository
`fossil add FILE`	Start tracking a file
`fossil commit -m "Meaningful message"`	Record a check-in with a message
`fossil diff`	Show textual differences of uncommitted changes
`fossil branch`	List branches
`fossil branch new NAME trunk`	Create a new branch from trunk
`fossil update trunk`	Switch the checkout to the tip of trunk
`fossil update CHECKIN_ID`	Make the checkout match a historical check-in
`fossil merge BRANCH`	Merge a branch into the current checkout (then commit)
`fossil revert FILE`	Discard uncommitted changes to a file
`fossil tag list`	List tags
`fossil pull`	Fetch changes from the remote repository
`fossil push`	Send changes to the remote repository
`fossil remote`	Show the configured remote repository URL
`fossil ui`	Start the local web interface
`fossil alerts send`	Send queued email alerts (needed manually before automatic processing was working)

Appendix B: Screenshot Evidence Index

This index lists every figure, whether an image has been supplied, and where the evidence should come from. The “Required item” column maps each figure to the team’s original list of 23 required screenshots. Placeholders are bordered boxes; to replace one, click inside it, delete the text and insert the picture.

Table 29. Figure register

Figure	Title	Status	Evidence from	Required item
Figure 1	RAG system architecture diagram	Placeholder	Team (draw in a diagramming tool; this is a design diagram, not captured evidence)	22
Figure 2	DocuMind application home page	Placeholder	DocuMind application	19
Figure 3	Document upload and processing	Placeholder	DocuMind application	20
Figure 4	Question-answering interaction	Placeholder	DocuMind application	21
Figure 5	Original Fossil web interface	Supplied (embedded)	Fossil repository (before customisation)	1
Figure 6	Fossil user list	Supplied (embedded)	Fossil repository (Admin → Users)	2
Figure 7	Repository timeline	Placeholder	Fossil repository (Timeline page)	3
Figure 8	Branch creation and feature check-in	Placeholder	Fossil repository (Timeline / check-in page)	6
Figure 9	Merge into trunk	Placeholder	Fossil repository (Timeline page)	7
Figure 10	Ticket management page	Placeholder	Fossil repository (Tickets page)	8
Figure 11	Tags / release identification	Placeholder	Fossil repository (Tags page or terminal)	9
Figure 12	Fossil wiki and documentation	Placeholder	Fossil repository (Wiki page)	10
Figure 13	Windows collaboration evidence	Placeholder	Windows teammate (PowerShell or Fossil UI)	4
Figure 14	Linux collaboration evidence	Placeholder	Linux teammate (terminal)	5
Figure 15	Custom DocuMind dashboard overview	Supplied (embedded)	Fossil repository (custom skin, Niranjan Santhosh)	11, 12
Figure 16	Quick Actions	Supplied (embedded)	Fossil repository (custom skin, Niranjan Santhosh)	12
Figure 17	DocuMind services and RAG workflow	Placeholder	Fossil repository (custom skin, Niranjan Santhosh)	13
Figure 18	Documentation section	Supplied (embedded)	Fossil repository (custom skin, Niranjan Santhosh)	12
Figure 19	Repository Activity	Supplied (embedded)	Fossil repository (custom skin, Niranjan Santhosh)	12
Figure 20	Repository overview and status	Supplied (embedded)	Fossil repository (custom skin, Niranjan Santhosh)	14, 15

Figure	Title	Status	Evidence from	Required item
Figure 21	Project team, recent activity and version	Supplied (embedded)	Fossil repository (custom skin, Niranjan Santhosh)	15
Figure 22	Fossil email-server settings	Placeholder	Notification setup (Harisankar C Mooss)	—
Figure 23	Test email in Gmail	Placeholder	Notification setup (Harisankar C Mooss)	—
Figure 24	Check-in notification email	Placeholder	Notification setup (Gmail inbox)	16
Figure 25	Ticket notification email	Placeholder	Notification setup (Gmail inbox)	17
Figure 26	Administrator subscriber configuration	Placeholder	Fossil repository (Alerts / Subscribers page)	18
Figure 27	Testing and result evidence	Placeholder	DocuMind application / terminal	23